

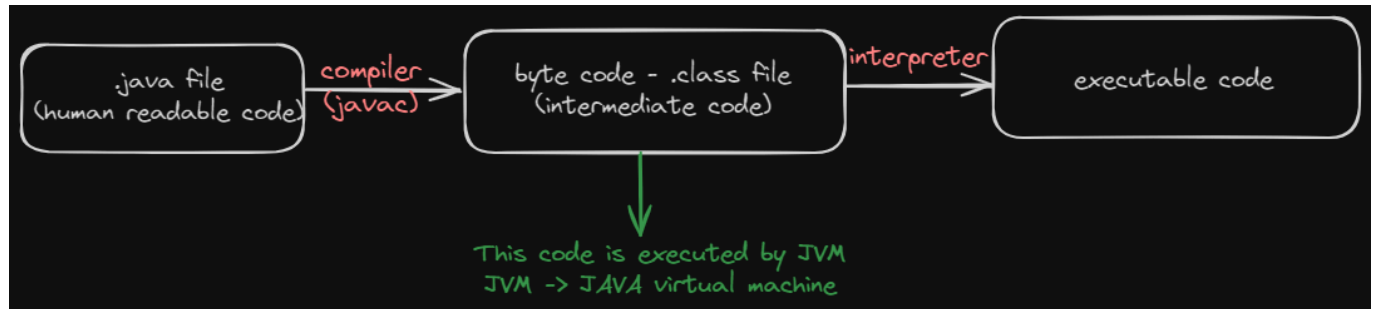
1. Java Working

Java is a programming language which is compiled as well as interpreted. It also uses the methods of Object-Oriented programming.

Code execution

Java code execution usually takes place in 3 steps

[java-code-execution.excalidraw](#)

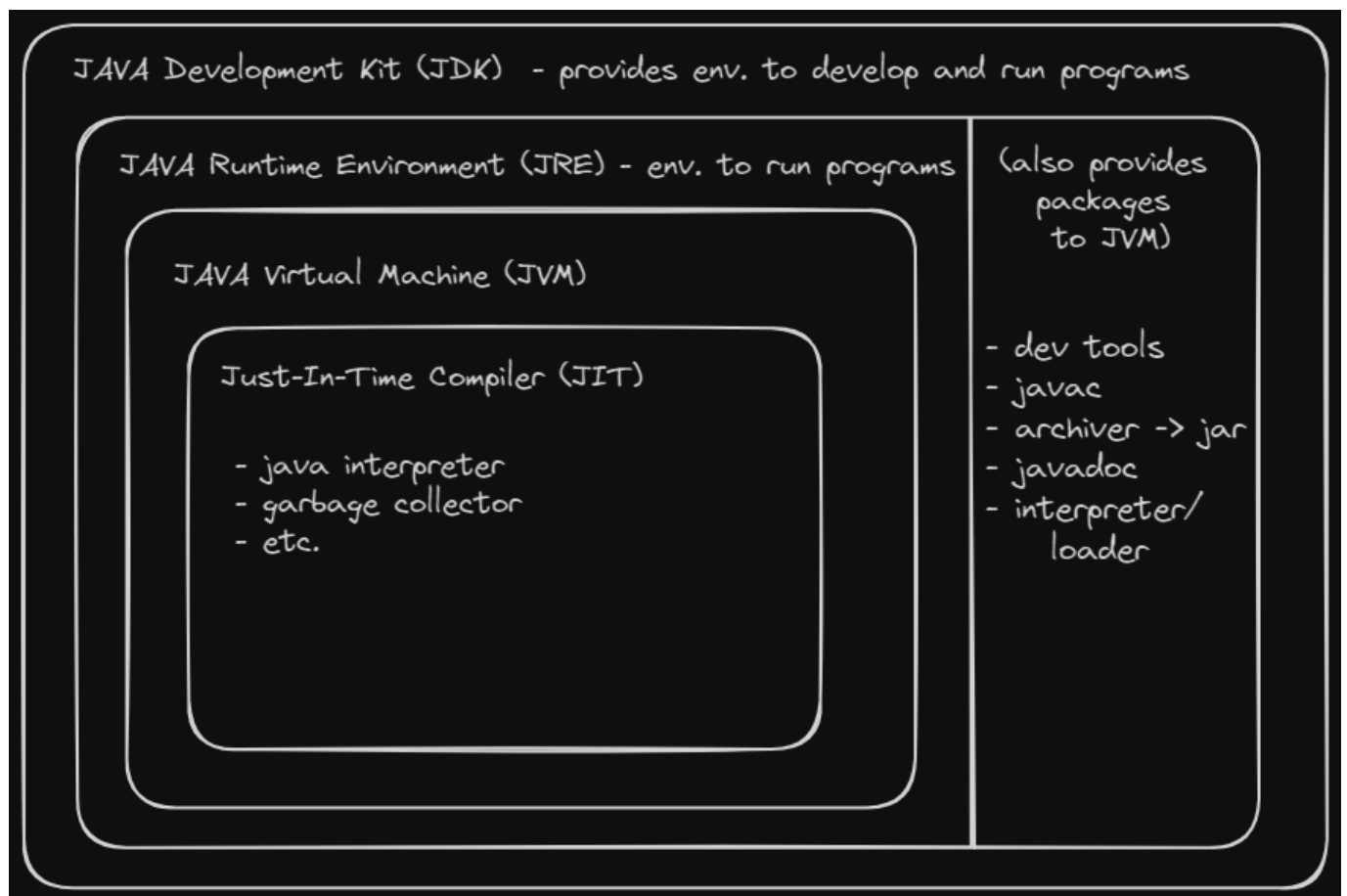


- The .java file is something a user creates, this contains all the classes and functions.
- The compiler (javac) compiles this code into an **OS independent** code called **byte code**. This is not the final executable code, this is what we call an intermediate code.
 - OS independent implies that the byte can be compiled into an executable file on any operating system (no matter what the inner architecture is).
- The interpreter (JVM - JAVA virtual machine -> OS dependent) converts the byte code into the **Machine code** (just a bunch of 0's and 1's). The interpreter converts it **line by line**.
 - This machine code however is **OS dependent**.

2. Java Environment

The Java code requires a specific environment to just even convert the source code into byte code.

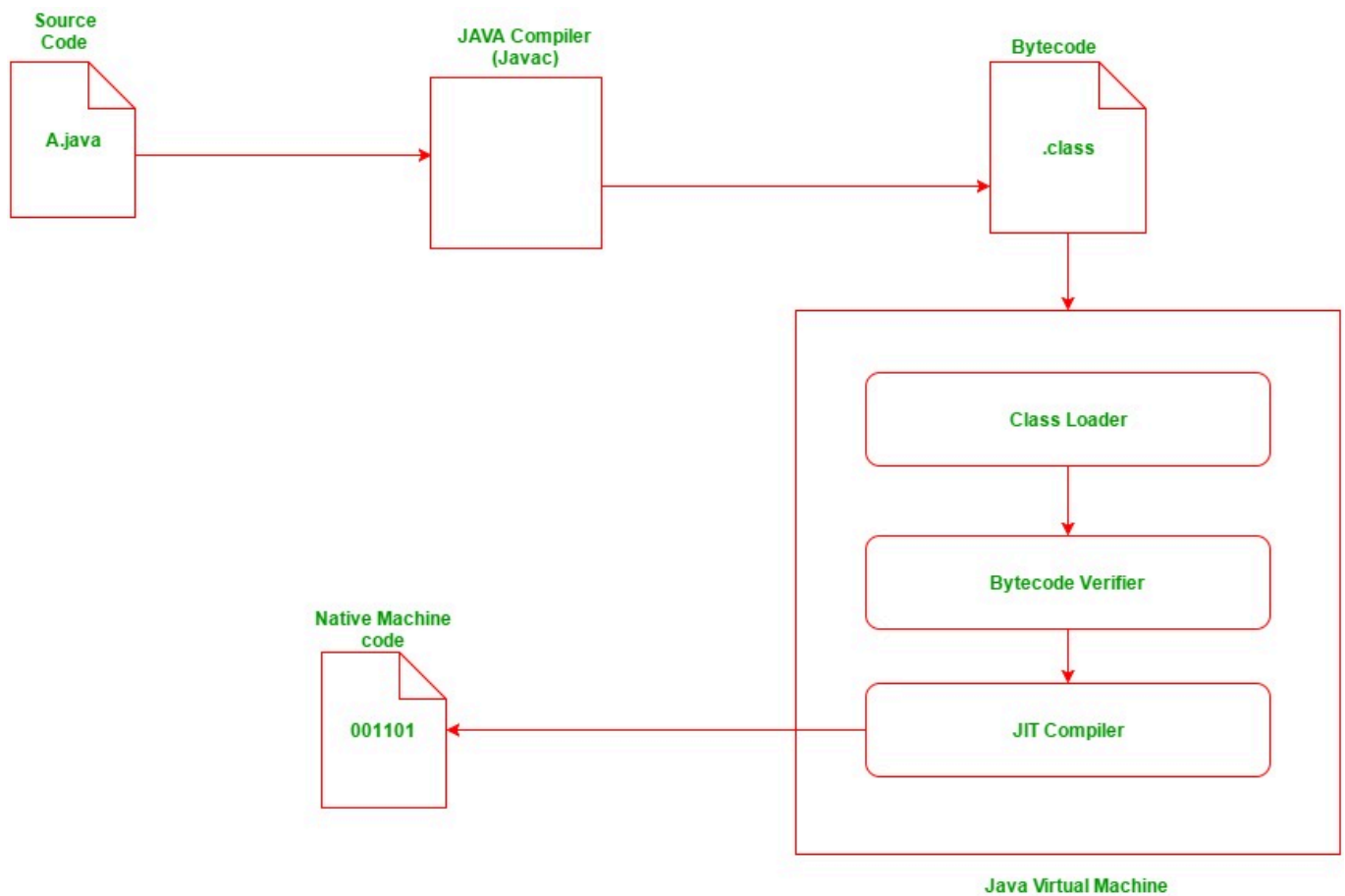
This environment is provided by the Java Development Kit.



- JAVA Development Kit (JDK)
 - This kit provides the main environment which allows the user to develop and run JAVA programs.
 - It contains a few tools/environment which help in development.
- JAVA Runtime Env (JRE)
 - This environment is strictly meant for running programs.
 - It contains libraries and tools that is used by the JVM to execute the programs.
 - Note that the actual content (code) isn't present in the JRE, it's present in the JVM
- JAVA Virtual Machine (JVM)
 - JVM is where the byte code is converted into the executable file.
 - JIT Compiler
 - This compiler is present to help the JVM to provide it with repeated-code bases.
 - The JVM is an interpreter which converts the code **line by line** into the machine code. To prevent repeated conversion of the same code, the JIT provides the JVM with the machine code.

3. Compile Time and Runtime

JAVA involves a two step process which includes compilation and then interpretation.



Compilation

In this process, the `javac` compiler converts the source code into a byte code which has the extension of `.class`

Execution (Runtime)

The class file generated is OS independent. This class file is then sent into the JVM to convert it into the machine code.

There are 3 stages of conversion involved in the JVM

1. Class Loader
2. Byte Code Verification
3. JIT Compiler

Class Loader

This step involved the "ClassLoader" which loads all the necessary and referenced classes needed to execute the program. This is then converted to the bytecode.

Class Loading consists of 3 major steps:

1. Loading
 - Reading of the `.class` file and generating its binary data
 - This data (class) is created as an instance of the class (Object) in the heap
2. Linking
 - JVM verifies the `.class` file

- Allocates memory for class variables and default values
- replace symbolic references from the type with direct references.

3. Initialization

- All static variables are assigned with their values defined in the code and static block.
- JVM contains the stack and heap memory locations.

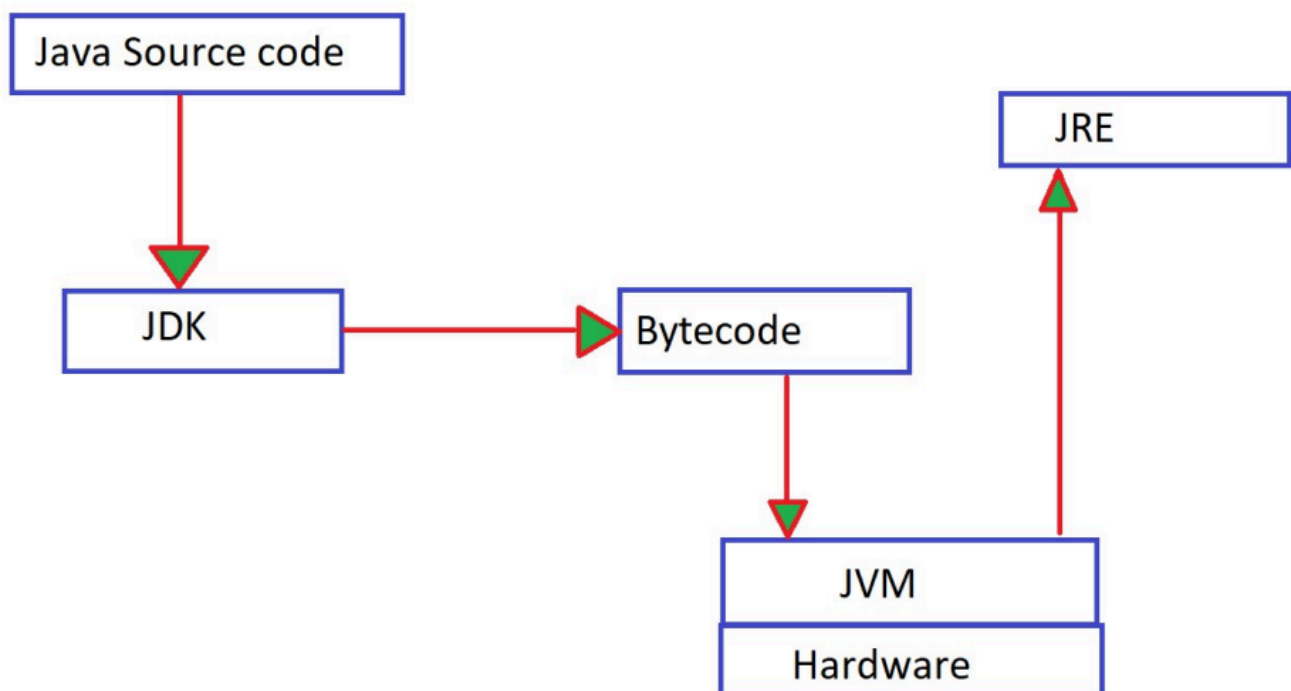
Byte Code verification

Once the byte code is generated, JVM sends the code out for verification, where any errors or illegal practices are checked by the **bytecode verifier**.

Just-In-Time Compiler

The final stage involves the conversion of the verified byte code in the machine code (0's and 1's). The JIT helps in providing the native code to the JVM instead of the JVM interpreting the same lines of code again and again.

This can lead to performance gains in the execution speed unless methods are executed less frequently.



4. Functions in Java

5. Arrays & ArrayList

Arrays are a data structure used to store multiple single data-type values together.

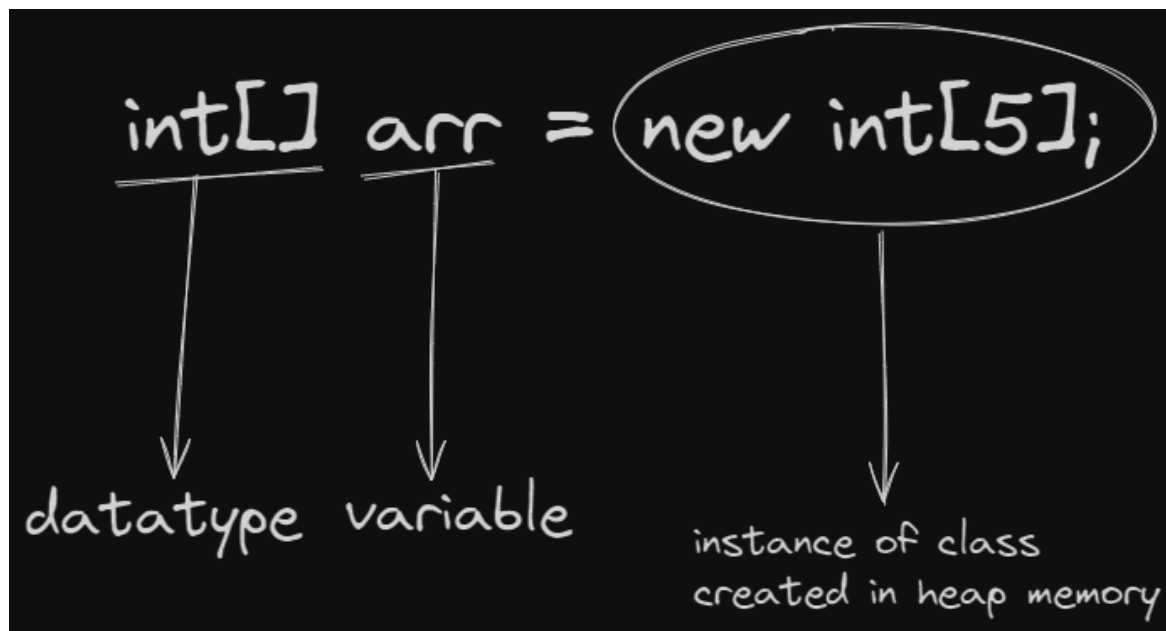
Syntax of Arrays ->

```
datatype[] var_name = new datatype[size];
```

When a new array is created, first the **declaration** takes place in **compile** time and the **initialization** takes place in **runtime**.

By default all values during initialization are set to 0 or null. (null is a default value given to reference vars.)

Size is allocated in runtime, this is **Dynamic memory allocation**



Are arrays in JAVA continuous?

In C/C++ there is a concept of pointers, which basically means we can get the address of anything.

This allows us to have continuous addresses allotted to arrays. But in JAVA, there is **no concept of pointers**.

Why?

1. Arrays are objects in heaps
2. JSL mentions that heap objects are NOT continuous.
3. Heap is also the **runtime** data area. (Dynamic memory allocation).

It's continuity depends on the JVM.

What is new?

```
new int[size];
```

`new` is used to create an instance of the class, i.e. Creation of an object of given size.

ArrayList

As we saw above, the size of the array is fixed and we need to always provide the size. But what if we need to store something in an array, but we don't know its size, it could be possible in some cases that we want to store something that has an ever growing size.

This is where an ArrayList comes into picture. It allows us to store items into an array, without actually defining its size.

How is it possible to have a data structure that doesn't have a definite size? Well, an arraylist doesn't actually have indefinite size, **it's definite**.

Internally, a `ArrayList` has a defined size. When we want to increase the size, the old array is copied into a new array, with double (approx) the size. It takes about $O(1)$ time to copy and increase the size. (that's just how it works).

Note that the array is not actually doubled.

```
// formula given in java docs
int newCapacity = ArraysSupport.newLength(oldCapacity,
                                         minCapacity - oldCapacity, /* minimum growth */
                                         oldCapacity >> 1           /* preferred growth */);
```

6. Null in java

Null is nothing but a special literal type that can be of any reference type. In simpler terms it means that it be a reference to anything in absence of a value. It's **neither an object nor a type**.

Each class variable, instance variable, or array component is initialized with a *default value* when it is created: For all reference types, the default value is `null`.

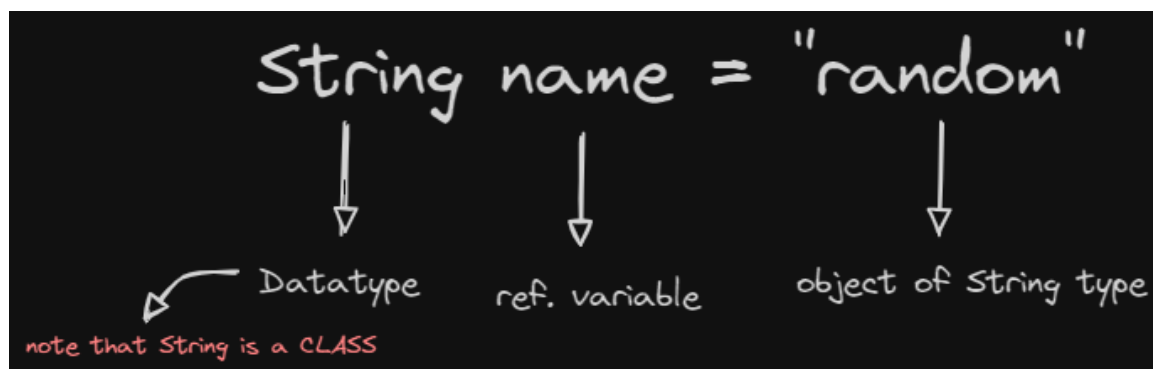
Generally speaking, `null` are used as a special value to signify:

- Uninitialized state
- Termination condition
- Non-existing object
- An unknown value

How is it represented in the memory?

In Java? None of your concern. And it's best kept that way.

7. Strings & StringBuilder in Java



String Pool & Immutability

What exactly are these two terms? Lets understand with an example.

Lets say we have two variables pointing to the same thing.

```
String a = "test";
String b = "test";
```

Does 'a' & 'b' point to the same String object? Or are two different string objects created? The answer is no.

JAVA creates something called a **String Pool** when we first create a string object. In our example, when we created a string object pointed by `a`, a String pool memory was created inside the heap. This string pool contains our string object (in this case 'test').

But why do we exactly need this pool of strings???? Recall how if we had two variables pointing to the same array, and if we made changes to the array w.r.t to one variable, the other variable pointing to the array was also affected.

To prevent this from happening, by default in JAVA strings are **Immutable**. This was done to improve security, as in a normal program we could have multiple variables pointing to the same string object.

To check this, we can do a simple comparison between the strings using the `==` operator. If the variables point to the same string object, the operator WILL return true.

```
System.out.println(a == b) // true
```

BUT what if we need to make two **different variables, point to two different string object**.

```
String a = new String("test");  
String b = new String("test");
```

Simply use the new keyword. What it does is simply create a new object with whatever value is mentioned inside the brackets, BUT the objects are created **outside of the string pool**.

So now if we run the `==` operator on the objects, it will return false. But we can use a method of the String class.

```
System.out.println(a == b); // false - checks if the OBJECT is same.  
System.out.println(a.equals(b)); // true - only checks the VALUES.
```

Slight note here, internally **String** class is a byte sized character array, but we can't manipulate and use it just like the char Array, String has its own set of methods.

This was until JAVA 8, from JAVA 9 onwards, a new representation is provided, called **Compact Strings**, this will appropriately choose between char[] and byte[]. This reduces the amount of heap memory significantly.

Interning

This is nothing but a terminology used when Java stores only one copy of a string literal in the pool.

Lets say we do the following

```
String constantString = "interned Baeldung";  
String newString = new String("interned Baeldung");  
assertThat(constantString).isNotSameAs(newString);
```

If we check the equality of `constantString` and `newString`, we will get false.

Java allows us to manually "intern" the string objects using the intern method.

```
String internedString = newString.intern();  
assertThat(constantString).isSameAs(internedString);
```

In version before JAVA 7, string pool were actually stored in a **PermGen** space instead of the heap, this would cause OutOfMemory errors if the user did interning with too many strings. But from JAVA 7 onwards, string pool are stored in the heap which also helps in garbage collection for unreferenced pools.

Printing

Generally in java, everything that is printed is actually just a string.

How?

When we do println, what exactly happens to the values inside the brackets? Lets see.

The internal working of println calls a function called "**valueOf**". This valueOf function in returns calls another function called "**toString**".

So the (overview) flow of function calling is,

```
println -> valueOf -> toString
```

Inside the valueOf function, there is a check done to see if the value received is null or not, if it is a null value, "null" is simply returned or if it isn't null, the **toString** method is called.

This works properly for normal data types, but what if we pass an array

```
System.out.println(new int[]{1, 2, 3, 4, 5});
```

This will return some random hexxed value to us, which not of much use to us. Instead what we can do is, do **function overloading** of the toString method. In our case, **Arrays** has it's own toString method which returns us the array and also pretty prints it.

```
System.out.println(Arrays.toString(new int[] {1, 2, 3, 4, 5}));
```

String concatenation

In JAVA, the **+** operator is overloaded natively to allow concatenation of strings or complex datatypes with strings

Note that two complex datatypes can't be concatenated, one **must** be a string

```
System.out.println('a'+'b');  
System.out.println("a"+"b");  
System.out.println("a"+1);  
System.out.println("Kunal" + new ArrayList<>()); // System.out.println(new Integer(0) + new  
ArrayList<>()); // error
```

One big problem with using normal strings is that, say we want to mutate a string using the **+** operator, everytime we use it, a new object is created. This is a memory issue and also a costly method (n^2 complexity).

This is solved with another datatype **StringBuilder**.

String Builder

The String Builder Objects are like String objects except they can be modified. StringBuilders internally are treated just like variable-length arrays that contain a sequence of characters.

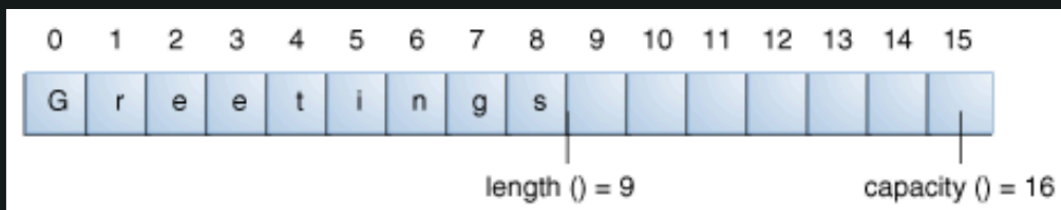
The StringBuilders provide better performance in certain use cases (as discussed above).

The capacity of a SB (string builder) is initiated to 16 (if not specified).

For example, the following code

```
// creates empty builder, capacity 16
StringBuilder sb = new StringBuilder();
// adds 9 character string at beginning
sb.append("Greetings");
```

will produce a string builder with a length of 9 and a capacity of 16:



Note: You can use any `String` method on a `StringBuilder` object by first converting the string builder to a string with the `toString()` method of the `StringBuilder` class. Then convert the string back into a string builder using the `StringBuilder(String str)` constructor.

8. OOPS in JAVA 1

Introduction

Object-Oriented Programming is a methodology or paradigm to design a program using classes and objects.

What are classes and objects? Imagine classes as a template, these are not real things (analogy to the real world). For example, think of "Human" as a class, a "Human" isn't exactly real, but a type of "human" exists in various forms. This "human" is a **class** and the type of humans (or instances of humans) are "objects".

To properly define these, Classes are a **blueprint** which are just logical identities. Objects are nothing but an instance of a class, the objects have **State, Identity & Behavior** associated with them.

Classes don't take up space in memory but the instance of a class, i.e the object takes up space in memory.

Create a Class

How to create a class? First off why do we need one. With the help of a class we can make new data structures to satisfy our needs. Classes also allow us to do various other cool things which are covered later in OOP's.

```
class Xyz{
    // note that classes start with a capital latter
    int roll;
```

```
String name;  
}
```

Above is a basic example of a class, it can be as basic as the above example, or very complex for larger scale applications.

Create an Object

How do we create an Object?

```
XYZ random = new XYZ();
```

`new` is something we discussed earlier, it's used to instantiate a class or create a new instance of a class. `XYZ()` is nothing but a **constructor** that helps call the class? No. This function call is actually used to construct the object using the class properties, every time we build an object, the constructor is always called no matter what.\

The left part of the statement happens at **compile time** and the right side happens at **runtime**.

In the above example, `random` is pointing to an object / is dynamically assigned memory at runtime.

But if we do the following,

```
XYZ random;
```

There is no space allotted to `random`.

Constructor

The default constructor is `constructor()`. We can (overload this/change) the constructor by writing our own constructor

```
class XYZ {  
    int roll;  
    String name;  
  
    XYZ(){  
        // default  
    }  
    XYZ(int roll, String name){  
        // constructor overloading  
        this.roll = roll;  
        this.name = name;  
    }  
}
```

The `this` keyword refers to the object that is calling the function/instructor.

```
XYZ random = new XYZ(10, "name");
```

After compilation, when the program runs, JAVA automatically replaces `this` with the object name, i.e. in this case, `random`.

```
random.roll = 10;
random.name = "name"; // this is what essentially happens
```

Lets say we create a object, and we create another object pointing to the first one, both the objects point to the **same** instance in heap. Meaning we changes in one reference can also be seen in the first reference.

Wrapper Classes

You might have notices there are two ways to create integers in JAVA,

```
int a = 10;
Integer b = 10;
```

`int` is nothing but a primitive data type stored in the stack whereas `Integer` is a **wrapper class**. A wrapper classes gives us access to many function that aren't available with the primitive data type.

Note that Primitives are **passed by values** (only passing value) & objects are also **passed by values but values of the reference variable are passed** which means that the a reference variable is created which is pointing to the object.

- When you pass a primitive type (like `int`, `float`, or `boolean`) to a method, Java creates a copy of the actual value

```
public class PassByValue {
    public static void main(String[] args) {
        int number = 5;
        modifyValue(number);
        System.out.println("Original number: " + number); // prints 5
    }

    public static void modifyValue(int num) {
        num = 10; // This change does not affect 'number'
    }
}
```

- When you pass an object to a method, you are passing the value of the reference variable that points to that object
- **Reassignment Does Not Affect Original Reference:** If you try to reassign the reference within the method to point to a new object, it will not change the original reference outside the method.

```
class Balloon {
    private String color;

    public Balloon(String color) {
        this.color = color;
    }

    public String getColor() {
        return color;
    }
}
```

```

    public void setColor(String color) {
        this.color = color;
    }
}

public class ObjectPassByValue {
    public static void main(String[] args) {
        Balloon balloon = new Balloon("Red");
        System.out.println("Before change: " + balloon.getColor()); // prints Red
        changeColor(balloon);
        System.out.println("After change: " + balloon.getColor()); // prints Blue
    }

    public static void changeColor(Balloon b) {
        b.setColor("Blue"); // Modifies original object's state
    }
}

```

```

public class ReferenceReassignment {
    public static void main(String[] args) {
        Balloon balloon = new Balloon("Red");
        System.out.println("Before reassignment: " + balloon.getColor()); // prints Red
        reassignBalloon(balloon);
        System.out.println("After reassignment: " + balloon.getColor()); // still prints Red
    }

    public static void reassignBalloon(Balloon b) {
        b = new Balloon("Green"); // This does not affect 'balloon' in main()
    }
}

```

FINAL keyword

This keyword is used to disable the user to modify your content present in the object. You can also use the `final` keyword in a method declaration to indicate that the method cannot be overridden by subclasses

```

final int foo = 3.14;

final class Xyz {
    Xyz(){
        System.out.println("Hello World");
    }
} // cant be extended by any other class

```

Final works on **Classes, methods and other data types**

Finalisation

When an object is dereferenced, the JAVA GC automatically cleans it (unlike in C/C++ we need to use the destructor method).

Here JAVA gives us a method called finalisation where we can call this method **only after** the object is removed by the GC.

9. OOPS IN JAVA 2

Lets say we create a object and we try to print the object. As we learnt earlier, the `println` will call `valueOf` and that will call the `toString` method.

Now if our object has a `toString` method, that's well and good, but if it doesn't, then it will call the default `toString` method.

Packages

A package is a namespace that **organizes** a set of related classes and interfaces. Conceptually you can think of packages as being similar to **different folders on your computer**.

Packages allows us to use code from different folders in a single place without actually writing the whole code again.

Basically we can import another function from a package in our current working package. How does JAVA know where to look tho? During runtime it first looks for the function in the current working directory and then in the other directories.

The Java platform provides an enormous class library (a set of packages) suitable for use in your own applications. This library is known as the "Application Programming Interface", or "API" for short. Its packages represent the tasks most commonly associated with general-purpose programming.

How Java Knows Where to Look for Functions and Classes

1. Packages as Namespaces:

- Packages group related classes and interfaces, helping avoid naming conflicts.
- Each package corresponds to a directory structure in the file system.

2. Importing Classes:

- Use import statements to include classes from other packages, simplifying access without fully qualified names.
- Example: `import java.util.ArrayList;`

3. Classpath:

- The classpath tells the JVM and compiler where to find user-defined classes and packages.
- It can include directories, JAR files, and ZIP files.

4. Class Loading Process:

- During execution, the JVM looks for the main class specified in the command line.
- If that class references others, it searches for them in the classpath.

5. Order of Resolution:

- **Current Directory:** First checks the current working directory.
- **Classpath Directories:** Then searches through directories specified in the classpath.
- **JAR Files:** Finally checks within JAR files listed in the classpath.

Static Keyword

The Static keyword is something that allows us to add a common property to our class. This property is something that isn't related to the object, it's **object independent**.

So now let's say we create an object as such

```
public class Human {  
    int age;  
    static int population;  
}
```

So to actually use this static variable inside any method/constructor we will not use `this` keyword, instead we will use the class name (here `human`).

If we try to access the static variable using `this` keyword, the compiler at runtime will check whether the object consists of the variable, if not, it will check its class. So using the class name instead of `this` is just a convention.

One of the most famous static methods we use is `public static void main`. But why is the `main` function a static method?

1. Entry Point for JVM:

- The `main` method serves as the entry point for any Java application. The Java Virtual Machine (JVM) needs to call this method to start executing the program.

2. No Object Required:

- Since the JVM does not create an instance of the class before calling the `main` method, it must be static. This allows the JVM to invoke the `main` method directly using the class name without needing an object.

3. Avoiding Ambiguity:

- If `main` were not static, the JVM would need to instantiate the class first, which could lead to ambiguity, especially if there are multiple constructors. This would complicate how the JVM determines which constructor to use.

4. Memory Efficiency:

- Declaring `main` as static avoids unnecessary memory allocation that would occur if an object were created solely to call this method.

5. Standardized Signature:

- The signature of `main` (i.e., `public static void main(String[] args)`) is standardized across all Java applications. This consistency simplifies how IDEs and tools detect launchable classes.

Consequences of Not Using Static in main

If you do not declare the `main` method as static, you will encounter issues during execution:

- The JVM will throw a runtime error stating that it cannot find the main method.
- Example of an incorrect implementation:

```
class MyClass {  
    public void main(String[] args) { // Not static  
        System.out.println("Hello World");  
    }  
}  
  
public class Main {
```

```

    public static void main(String[] args) {
        MyClass obj = new MyClass();
        obj.main(args); // This won't be called by JVM directly
    }
}

```

- Output: An error indicating that the main method is not found.

Characteristics,

- **Memory Management:** Static members are allocated memory only once during class loading.
- **Access:** Can be accessed without creating an instance of the class.

So now let's say we want to use another function inside our main function, we usually write `static` to this function. Why?

Something that is `not static`, actually belongs to an object and something is `static`, doesn't belong to an object

When working with static functions, you can only call other static methods within them, since static methods do not depend on an instance of the class.

Non-static (or instance) methods, on the other hand, can access both static and non-static members because they have access to the class instance.

TLDR, you can't access non-static stuff without referencing in a static context

```

package oops;

/**
 * staticExample
 */
public class staticExample {
    public static void main(String[] args) {
        System.out.println("Hello world");
        fun2(); // no error

        // Test one = new Test(0); //! error -> can't access non-static instance in static
        // instance.

        Test2 two = new Test2(0); // no error
    }

    static void fun2(){
        // fun(); //! error -> Cannot make a static reference to the non-static method fun()
        // from the type staticExample
        System.out.println("inside fun 2");

        // to use a non-static function
        // we need to first create its instance
        staticExample obj = new staticExample();
        obj.fun();
    }
}

```

```

}

void fun(){
    System.out.println("inside fun");
}

void fun3(){
    fun(); // no error
}

// class Test {
//     int num;
//     public Test(int num){
//         this.num = num;
//     }
// } //!-> gives error when called in main

class Test2 {
    int num;
    public Test2(int num){
        this.num = num;
    }
}

```

- `fun2()` is a static method, so it **belongs to the class** `staticExample`. It can be called without creating an instance of `staticExample`, like in the `main()` method where you simply call `fun2()` directly.
- `fun()` is a non-static method, so it **belongs to an instance (object)** of the class `staticExample`. To call `fun()`, you need to first create an object of `staticExample`, as seen in `fun2()` where `obj.fun()` is used.

Notice that calling `fun()` inside of `fun3` doesn't throw an error. This is because in the end everything gets called by a static function and JAVA knows this. So it will automatically create an object for it.

When we try to create an instance of `Test` in `main`, it gives an error as `Test` now depends on the `staticExample` class.

Static stuff is resolved during runtime (as static doesn't depend on objects and objects are resolved during compile time).

Singleton Classes

Singleton classes satisfy a very particular use-case. If we ever want to limit a class to a **single object** or a single instance we use singleton classes.

```

package oops.singleton;

// Class based singleton

public class SingletonClass {

```



```

private SingletonClass(){

    System.out.println("Singleton Class instantiated");

}

private static SingletonClass instance;

public static SingletonClass getInstance(){

    if(instance == null) instance = new SingletonClass();

    return instance;

}

}

```

10.OOPS IN JAVA 3

Inheritance

This is something similar to real life inheritance, like money, genetics etc. inheritance. The children inherit stuff from parents and can use these inherited properties.

In OOPs, we have a base class, from which we can derive a new **child class**, this child class can have all access to the variables/methods in the base class but not the other way around.

The **extends** keyword allows us to perform inheritance in JAVA.

```

class Base{
    //something
}

class Child extends Base{
    // can access something
    // and something else
}

```

We can perform multiple inheritances with multiple child classes.

Lets say we do something as follows

```

Base var = new Child(); // Parent to child reference
// Child var2 = new Base(); // Error

```

Now will we be able to access all the child class variables? No. This is because the type of the object is defined by its **reference type**. (i.e the LHS side, here it's **Base**).

The above example only works for a **Parent to child reference**, not the other way around, this is because the stuff present in the child classes may or may not be present in the parent class which may cause issues while calling the constructor.

```
// Parent Class
class Parent {
    void display() {
        System.out.println("This is the Parent class.");
    }
}

// Child Class
class Child extends Parent {
    void display() {
        System.out.println("This is the Child class.");
    }

    void childMethod() {
        System.out.println("Child specific method.");
    }
}

// Main Class
public class Main {
    public static void main(String[] args) {
        // Creating an instance of Child but referencing it with Parent type
        Parent parentRef = new Child(); // Parent reference holding Child object

        // Calling method from Parent class
        parentRef.display(); // Output: This is the Child class.

        // The following line would cause a compile-time error because
        // parentRef cannot access child-specific methods
        // parentRef.childMethod(); // Uncommenting this will cause an error

        // To access child-specific methods, you would need to downcast
        if (parentRef instanceof Child) {
            Child childRef = (Child) parentRef; // Downcasting
            childRef.childMethod(); // Output: Child specific method.
        }
    }
}
```

reference is the LHS part.

Feature	Upcasting	Downcasting
Definition	Converting child to parent reference	Converting parent to child reference
Casting Type	Implicit	Explicit
Access	Can access parent methods only	Can access both parent and child methods

Feature	Upcasting	Downcasting
Risk	Safe; no exceptions	Risk of <code>ClassCastException</code>

Super Constructor

JAVA provides a `super()` method which allows us to call the parent class constructor. In the child class we can accept the appropriate params for the parent & child class constructors and then we can simply call `super` to call the parent class constructor.

Note that each class has its own super method, even the top-most base class will have its own super method. **The super method will call the Object class.**

The super method also allows us access methods/variables present in the parent class by `super.something`.

Multiple inheritance is not supported in JAVA, it can be implemented by something called **interfaces**.

Hierarchical Inheritance in Java

Hierarchical inheritance is a type of inheritance in object-oriented programming where multiple classes (child classes) inherit from a single parent class (base class). This structure allows for code reusability and modularity, as common functionality can be defined in the parent class and shared among the child classes.

Key Features of Hierarchical Inheritance

1. **Single Base Class:** There is one parent class from which multiple subclasses derive.
2. **Code Reusability:** Common methods and properties can be defined in the parent class, reducing redundancy in child classes.
3. **Customization:** Each child class can extend or override methods from the parent class to provide specific implementations.

Advantages

- **Modularity:** Promotes a clear structure, making it easier to manage code.
- **Reduced Redundancy:** Shared functionality in the parent class minimizes code duplication.
- **Flexibility:** Child classes can have their own unique methods while still benefiting from inherited properties.

Disadvantages

- **Tight Coupling:** Changes in the parent class can affect all child classes, which may lead to unintended consequences.
- **Complexity:** As the hierarchy grows, understanding the relationships between classes can become challenging.

Example of Hierarchical Inheritance

Here's a simple example demonstrating hierarchical inheritance:

```

// Parent Class
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

// Child Class 1
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

// Child Class 2
class Cat extends Animal {
    void sound() {
        System.out.println("Cat meows");
    }
}

// Child Class 3
class Cow extends Animal {
    void sound() {
        System.out.println("Cow moos");
    }
}

// Main Class to demonstrate hierarchical inheritance
public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        Cat cat = new Cat();
        Cow cow = new Cow();

        // All classes can access the method of the Animal class
        dog.sound(); // Output: Dog barks
        cat.sound(); // Output: Cat meows
        cow.sound(); // Output: Cow moos
    }
}

```

Explanation of the Example,

- **Animal Class:** This is the parent class with a method `sound()` that can be overridden by child classes.
- **Dog, Cat, and Cow Classes:** These are child classes that extend `Animal`. Each class provides its own implementation of the `sound()` method.
- **Main Class:** In the `main` method, instances of each child class are created, and their specific implementations of `sound()` are called.

Feature	Multilevel Inheritance	Hierarchical Inheritance
Structure	Chain of classes (A → B → C)	Multiple classes from one parent (A → B, A → C)
Levels	Can have multiple levels	Typically one level of subclasses
Inheritance	Each subclass inherits from its immediate superclass	All subclasses inherit from the same superclass
Use Case	Useful for creating a more specific hierarchy	Useful for sharing common functionality among different classes

Hybrid Inheritance

Single and multiple inheritance -> Not in JAVA -> Implemented in **Interfaces**

Polymorphism

Poly means **many**, morphism means **ways to represent**. Languages that support Polymorphism are called OOP's languages, the ones that don't support are called **Object Basis Language**.

Compile/Static Polymorphism

This type of morphism is obtained by **method overloading** which means methods having **same names** but **different parameters**. Why compile/static? This is because JAVA determines the method to be called at **compile time**.

Runtime/Dynamic Polymorphism

This is achieved by **method overriding**. This can be used when we want have a function that has the same name and parameter as its parent class except the body.

We can check if a method is overriding or not, we can simply write the `@Override` annotation above the function in the child class.

```
class Base{
    void fun(){
        // something
    }
}

class Child extends Base {
    @Override // no need to mention this explicitly
    void fun(){
        // something
    }
}
```

```
Child var1 = new Child();
var1.fun(); // calls the function in Child
```

The way this works internally is that, JAVA first checks for availability of the method in its reference type and then it checks in the Object type for the method.

```
Base var2 = new Child();
var2.fun() // first checks Base for the method and then executes the Child method
```

The above described functionality to find the correct method is called **upsetting**. This is how overriding works.

But how does JAVA do this? JAVA does something called **dynamic method dispatch**. This method allows JAVA to select which method to call during the runtime (i.e. when the program is running).

Now let's say that we add the **final** keyword to any of the functions in the base class, that are also overridden by the child class method.

Adding the **final** keyword lets JAVA know during compilation itself that the particular function won't be overridden and that this is the one to be called later on. This removes the extra step for JAVA to check which method to run. This is also called as **early binding**.

Late binding is what we discussed earlier (i.e. without the **final** keyword, which function to call is resolved in runtime).

We know that overriding depends on the Objects, so that means we can't override static methods as they are dependent on Classes.

In Java, static methods cannot be overridden in the same way that instance methods can. Instead, they exhibit a behavior known as **method hiding/static method dispatch**. (this is closely related to *early binding*)

Overriding and overloading can't be done on instances.

Encapsulation

Wrapping of the implementation of the data members/methods in a class. This is basically hiding of the data from the outside world.

Abstraction

Hiding the unnecessary details, but only showing the valuable information.

Difference b/w Encap & Abstraction

Abstraction is a design issue. Meaning we provide the user with only the functions (i.e. the function name to call) and the user doesn't care about the internal body of the function.

Whereas **Encapsulation is a implementation issue.** Which means the implementation part that is to be provided for Abstraction.

TLDR, Encapsulation works on the internal working and Abstraction focuses on the external working.

Data Hiding

Data hiding is the hiding of the data from outside users. This might sound similar to Abstraction but as we saw earlier Abstraction is a design level issue. Whereas here, data hiding is a security issue.

Data hiding is implemented to improve data security and we can use keywords such as **private**, **public** & **default**.

Now this might even sound similar to Encapsulation, but encapsulation is a design issue which means that we are trying to hide the internal working of the function. Whereas here, again, data hiding means increasing the security of the data.

Say we hide some data (this is data hiding), then we can use/get this data using getters & setters (this is encapsulation).

```
package oops;

public class sandbox {
    public static void main(String[] args) {
        Parent realParent = new Parent();
        realParent.cantOverride();

        Parent pref = new Child(); /* Implicit Upcasting
        pref.x(); // Child's x /* Overriding or Dynamic Method Dispatch
        // pref.z(); /* error
        pref.cantOverride(); //parent's cantOverride /* method hiding or Static method
        dispatch or static polymorphism

        Child cref = (Child) pref; /* Downcasting
        cref.z(); // Child's z
        cref.cantOverride(); // childs's cantOverride
    }
}

class Parent {
    public void x() {
        System.out.println("Parent's x");
    }

    public void y(int i) {
        System.out.println("Parent's int show()");
    }

    static void cantOverride() {
        System.out.println("parent's cantOverride");
    }
}

class Child extends Parent {
    public void x() {
        System.out.println("Child's x");
    }

    public void z(){
        System.out.println("Child's z");
    }

    static void cantOverride() {
        System.out.println("childs's cantOverride");
    }
}
```

11. OOPS IN JAVA 4

Access Modified

1. Private (default)
2. Public
3. Protected

These access modifiers modify how we can access/use the data in classes or packages.

Access Modifier	Class	Package	SubClass (same pkg)	SubClass (diff pkg)	World (diff pkg & not subclass)
public	+	+	+	+	+
protected	+	+	+	+	
private	+				
no modifier	+	+	+		

Some packages in JAVA

1. Lang
2. io
3. util (data structures and etc.)
4. applet (to create applets)
5. awt (to create gui)
6. net (to connect to the net)

12. OOPS IN JAVA 5

Abstraction

In JAVA, multiple inheritance is not allowed, instead we can do it using something called "interfaces". But what are interfaces?

Let us first understand a small concept of abstract classes. Lets say we have a class, where only a method is defined without a body. In this case, it would be the child class's responsibility to implement this method.

This is known as **abstract methods**. Any class having an abstract method should always be an **abstract class**.

Abstract classes can have their own constructors (not abstract) as well but you can't create objects of abstract classes without overriding the methods in the parent class.

The following table tells us what all things can abstractify in JAVA.

Type	Yes/No
static	no
constructor	no
methods	yes
normal classes	yes
final classes	no

Additional points on abstract classes:

- Abstract classes can have both abstract and non-abstract methods.
- They are used to define a common interface for subclasses.
- Abstract classes can have instance variables and static variables.

Example of an abstract class:

```
abstract class Shape {  
    abstract double area();  
    abstract double perimeter();  
  
    void display() {  
        System.out.println("This is a shape.");  
    }  
}
```

Interfaces

Now that we understood the concept of abstract classes, let's understand what interfaces are. Interfaces are like abstract classes in which the functions are **public & abstract**, and the variables are **static & final**.

Interfaces can **extend** each other

The interfaces are **implemented** and not extended. The interfaces don't follow any hierarchy while inheriting.

Since the lookup of methods happens at runtime, interfaces should be used carefully in performance oriented applications.

Additional points on interfaces:

- Since Java 8, interfaces can have default and static methods with implementation.
- Since Java 9, interfaces can have private methods.
- A class can implement multiple interfaces, achieving a form of multiple inheritance.
- Interfaces are used to define a contract that implementing classes must follow.

Example of an interface:

```
interface Drawable {  
    void draw();  
}
```

```

    default void display() {
        System.out.println("Displaying drawable object");
    }
}

```

Annotations

Internally these are also interfaces.

Additional information on annotations:

- Annotations provide metadata about code.
- They can be used to give instructions to the compiler, for runtime processing, or for compile-time processing.
- Common built-in annotations include `@Override`, `@Deprecated`, and `@SuppressWarnings`.
- Custom annotations can be created using the `@interface` keyword.

Example of a custom annotation:

```

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Test {
    String value() default "";
}

```

Functional Interfaces

- Introduced in Java 8, these are interfaces with a single abstract method.
- They are the foundation for lambda expressions in Java.
- Examples include `Runnable`, `Callable`, and `Comparator`.

Default Methods in Interfaces

- Introduced in Java 8 to enable interface evolution.
- Allow adding new methods to interfaces without breaking existing implementations.
- Provide a default implementation that can be overridden by implementing classes.

```

package oops;

public class interfaceDemo {

    public static void main(String[] args) {
        interfaceDemo temp = new interfaceDemo();
        Painter p1 = temp.new Painter(); /* or just make Painter class static
        p1.draw();
        p1.display();

        Driver d1 = temp.new Driver();
        d1.display();
    }
}

```

```

class Painter implements DrawFace{
    public void draw(){
        System.out.println("Painter is drawing");
    }

    // @Override
    // public void display() {
    //     System.out.println("Painter override.");
    // }
}

class Driver implements DrawFace, NoDrawFace{
    public void noDraw(){
        System.out.println("I can't Draw");
    }

    public void draw(){
        System.out.println("I told you i cant draw");
    }

    // @Override
    // public void display() {
    //     // TODO Auto-generated method stub
    // }
}

interface DrawFace {
    void draw();

    default void display(){
        System.out.println("This is the DrawFace.");
    }
}

interface NoDrawFace {
    void noDraw();

    default void display(){
        System.out.println("This is the NoDrawFace.");
    }
}
}

```

```

interfaceDemo.java:30: error: types DrawFace and NoDrawFace are incompatible;
    class Driver implements DrawFace, NoDrawFace{
    ^
class interfaceDemo.Driver inherits unrelated defaults for display() from types DrawFace and

```

NoDrawFace

1 error

```
package abstractionAndInterfaces;

public class Test {
    public static void main(String[] args) {
        // abstract concept
        Circle a = new Circle();
        a.draw();
        System.out.println(a.x);

        // interface concept
        Square b = new Square();
        b.draw();
    }
}

// abstract concept
abstract class Shape{
    abstract void draw();
    int x = 10;
    void Shape(){
        System.out.println("Shape constructor");
    }
}

class Circle extends Shape{
    void draw(){
        System.out.println("Drawing a circle");
        x = 20;
    }
}

// interfaces concept
interface drawable{
    void draw();
    // if we make the above draw function as private, static, final, protected, etc, we will
    // need to implement it
    int x = 10; // static final variable
}

class Square implements drawable{
    public void draw(){
        System.out.println("Drawing a square");
        // x = 20; // error: cannot assign a value to static final variable x
    }
}
```

This code demonstrates various OOP and interface concepts in Java, including abstract classes, interfaces, functional interfaces, and anonymous classes. Below are the key notes explaining the code:

1. Main Class (`Test`):

The `Test` class contains the `main` method, which acts as the entry point for the program. Several objects are created and different types of draw methods are called to demonstrate different concepts:

- **Abstract Class (`Shape`):** An object of class `Circle` (which extends `Shape`) is created, and the `draw` method is called. The value of the instance variable `x` is printed.
 - **Interface (`drawable`):** An object of class `Square` (which implements the `drawable` interface) is created, and the `draw` method is invoked.
 - **Functional Interface (`drawable2`):** An object of class `Rectangle` (which implements the `drawable2` functional interface) is created, and the `draw` method is invoked.
 - **Anonymous Class:** An anonymous class implementing the `drawable2` interface is created. Its `draw` method is overridden directly within the `main` method and invoked.
-

2. Abstract Class (`Shape`):

- **Abstract Class Definition:**
 - `Shape` is an abstract class, meaning it cannot be instantiated directly.
 - It contains an abstract method `draw()` which must be implemented by any concrete subclass (like `Circle`).
 - It also has an instance variable `x` initialized to 10.
 - **Constructor-like Method:**
 - The method `void Shape()` is written like a constructor but is not actually a constructor since it has a return type (`void`). This won't be called as a constructor, so be aware of this mistake.
 - **Class `Circle` :**
 - This class extends `Shape` and implements the abstract `draw()` method, which prints "Drawing a circle".
 - It modifies the inherited instance variable `x` to 20.
-

3. Interface (`drawable`):

- **Interface Definition:**
 - `drawable` is an interface containing a single method declaration `draw()`.
 - Any class that implements this interface must provide an implementation for the `draw()` method.
- **Interface Variables:**
 - The variable `x` in the interface is implicitly `static` and `final`. This means it is a constant and cannot be modified. It is initialized to `10`.
- **Class `Square` :**
 - This class implements the `drawable` interface and provides an implementation for the `draw()` method, printing "Drawing a square".

- Trying to modify `x` in `Square` would cause an error, since `x` is `static final` in the interface.
-

4. Functional Interface (`Drawable2`):

- **Functional Interface Definition:**

- `Drawable2` is annotated with `@FunctionalInterface`, which signifies that the interface must have exactly one abstract method. In this case, it has the method `draw()`.
- A functional interface can be implemented using lambdas or anonymous classes (as demonstrated in the code).

- **Class `Rectangle`:**

- This class implements `Drawable2` and provides an implementation for the `draw()` method, which prints "Drawing a rectangle".

- **Functional Interface Rule:**

- If you try to add more than one abstract method in the `Drawable2` interface, it will cause a compilation error because functional interfaces can only have one abstract method.
-

5. Anonymous Class:

- **Anonymous Class Implementation:**

- In the `main` method, an anonymous class is created that implements the `Drawable2` interface.
- The `draw()` method is overridden inside the anonymous class, and it prints "Drawing an anon".
- Anonymous classes are useful for creating one-time implementations of interfaces or abstract classes without formally creating a new class.

- **Abstract Class:**

- Abstract classes cannot be instantiated directly. They can have both abstract methods and concrete methods (with implementations). Subclasses must implement all abstract methods.

- **Interfaces:**

- Interfaces define a contract that implementing classes must follow. Interface variables are implicitly `static` and `final`, meaning they cannot be changed.

- **Functional Interface:**

- A functional interface can have exactly one abstract method. They are often used in conjunction with lambdas or anonymous classes.

- **Anonymous Class:**

- An anonymous class is an unnamed implementation of an interface or abstract class, created on the fly, typically for one-time use.
-

Since Java 8, there was attempt to make java less verbose. Now we learn about lambda expressions which help us make java less verbose.

```
// func interface anon class using lambda
drawable2 anon2 = () -> {
    System.out.println("Drawing an anon2");
};
anon2.draw();
```

13. OOPS IN JAVA 6

Generics

In a nutshell, generics enable *types* (classes and interfaces) to be parameters when defining classes, interfaces and methods. Much like the more familiar *formal parameters* used in method declarations, type parameters provide a way for you to re-use the same code with different inputs. The difference is that the inputs to formal parameters are values, while the inputs to type parameters are types.

```

package oops;

import java.util.Arrays;

public class CustomGenericArrayList<T> {
    private Object[] data;
    /*
     * Bytecode generated has no idea about 'T' during runtime
     * Hence Object[] and not T[]
     */
    private static int DEFAULT_SIZE = 10;
    private int size = 0;

    public CustomGenericArrayList(){
        this.data = new Object[DEFAULT_SIZE];
    }

    public void add(T num){
        if(isFull()) resize();
        data[size++] = num;
    }

    private void resize(){
        Object[] temp = new Object[data.length * 2];

        for (int i = 0; i < data.length; i++) temp[i] = data[i];

        data = temp;
    }

    private boolean isFull(){
        return data.length == size;
    }

    public T remove(){
        T r = (T) (data[--size]);
        return r;
    }

    public T get(int index){
        return (T)(data[index]);
    }

    public int size(){
        return size;
    }

    @Override
    public String toString() {
        return "CustomArrayList{" +
            "data=" + Arrays.toString(data) +
            ", size=" + size +
            '}';
    }

    public static void main(String[] args) {
        CustomGenericArrayList<Integer> c1 = new CustomGenericArrayList<>();
        c1.add(1);
        c1.add(2);
        c1.add(3);
        c1.add(4);
        c1.add(5);
        System.out.println(c1);
    }
}

```

14. Static and final keywords

1. Static Keyword

- **Definition:** The `static` keyword is used for class-level variables and methods. It allows members to be shared among all instances of a class, meaning they belong to the class itself rather than any specific object.
- **Characteristics:**
 - **Memory Management:** Static members are allocated memory only once during class loading.

- **Access:** Can be accessed without creating an instance of the class.
- **Example Usage:** Commonly used for constants, utility methods, and shared resources.
- **Example:**

```
public class Counter {
    static int count = 0; // static variable

    public Counter() {
        count++; // Incrementing the static variable
    }

    public static void displayCount() {
        System.out.println("Count: " + count);
    }
}

public class Main {
    public static void main(String[] args) {
        new Counter();
        new Counter();
        Counter.displayCount(); // Output: Count: 2
    }
}
```

2. Final Keyword

- **Definition:** The `final` keyword is used to declare constants. Once a variable is declared as `final`, its value cannot be changed. It can also be applied to methods and classes to prevent overriding or inheritance.
- **Characteristics:**
 - **Immutability:** Final variables cannot be reassigned after their initial assignment.
 - **Usage in Methods and Classes:** Final methods cannot be overridden, and final classes cannot be subclassed.
- **Example:**

```
public class Constants {
    final int MAX_VALUE = 100; // final variable

    public void displayMaxValue() {
        System.out.println("Max Value: " + MAX_VALUE);
    }
}

public class Main {
    public static void main(String[] args) {
        Constants constObj = new Constants();
        constObj.displayMaxValue(); // Output: Max Value: 100
        // constObj.MAX_VALUE = 200; // This would cause a compile-time error
    }
}
```

```
}  
}
```

3. Using Both `static` and `final` Together

When both keywords are used together, it indicates that the variable is a constant associated with the class itself. This means it can be accessed without an instance, and its value cannot be changed.

- **Example:**

```
public class MathConstants {  
    static final double PI = 3.14159; // static final variable  
  
    public static void displayPi() {  
        System.out.println("Value of PI: " + PI);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        MathConstants.displayPi(); // Output: Value of PI: 3.14159  
        // MathConstants.PI = 3.14; // This would cause a compile-time error  
    }  
}
```

Consequences of Using `static` and `final`

1. **Memory Efficiency:** Using `static final` for constants saves memory since only one copy exists for all instances.
2. **Immutability:** Ensures that critical values (like mathematical constants) remain unchanged throughout the program, enhancing reliability.
3. **Accessibility:** Allows easy access to constants without needing to instantiate the class, promoting cleaner code.

Summary

- The `static` keyword allows class-level access and shared resources among instances.
- The `final` keyword ensures immutability, preventing changes to variables, methods, or classes.
- Combining both creates constants that are accessible at the class level and cannot be modified, promoting good programming practices.

Citations:

- [1] <https://www.geeksforgeeks.org/static-keyword-java/>
- [2] https://www.w3schools.com/java/ref_keyword_static.asp
- [3] <https://www.digitalocean.com/community/tutorials/static-keyword-in-java>
- [4] <https://www.simplilearn.com/tutorials/java-tutorial/static-keyword-in-java>
- [5] <https://www.javatpoint.com/static-keyword-in-java>
- [6] <https://dspmuranchi.ac.in/pdf/Blog/Static%20variable%20and%20method.pdf>

- [7] <https://www.programiz.com/java-programming/static-keyword>
- [8] <https://www.cs.virginia.edu/~jh2jf/courses/cs2110/java-pass-by-value.html>